UNDERSTANDING PLACEMENT ON THE HPE CRAY EX SUPERCOMPUTER

INTRODUCTION

Note that there are various versions of this document, you should be reading the version appropriate for your system – this document is for a Slurm environment.

As you will know by now, there are various methods to verify how the processes and threads of an application are bound to hardware resources (specifically Linux cpus). Examples are the environment variables that enable affinity display in the CCE compiler, with OpenMP and the MPICH implementation. Slurm also has the capability to display binding, this is enabled by the verbose argument to the –cpu-bind option. Also note that the Cray documentation mentions a simple program (xthi) that can be used to check binding.

For these exercises you have the option to use a program `acheck` which is a C program designed to help users understand how Processing Elements (PE) and threads are launched onto the compute nodes and how they are individually bound to CPUs. It has a very easy to understand output format. `acheck` is a hybrid MPI/OpenMP program. Using a program to check affinity is very useful to verify application binding for a job script, just point to acheck in the srun command instead of the application.

COMPILATION

`acheck` can (and should) be built with all of the compiler tool chains you intend to use on the HPE Cray EX system, the Makefile will automatically build a binary specific to the `PrgEnv` module loaded. Unpack the acheck tar file and build the program with

```
make
```

With the PrgEnv-cray module loaded this will build

```
acheck-cray
```

EXECUTION

Also provided is a simple execution script, `run-acheck.slurm`, that can be used to submit the job to the scheduler. You will need to adapt it for site requirements for account, partition, qos and reservation or use sbatch command line arguments to supply those details if needed.

You will need to experiment with different Slurm options to set

- Number of tasks
- Number of threads
- Allow/Disallow use of SMT threads on a core
- cpu binding policy
- task distribution
- etc.

Remember to set OMP_NUM_THREADS appropriately.

For more information on how each of these options affects how the application is launched please refer to the accompanying lecture or

```
man srun
```

UNDERSTANDING THE OUTPUT

By default acheck will print a brief summary concerning how it was launched. e.g.

```
 % export OMP_NUM_THREADS=4
% srun -n32 --cpus-per-task=4 ./acheck-cray
MPI and OpenMP Affinity Checker v1.23

Built with compiler: CCE/Clang 14.0.2

There are 32 MPI Processes on 1 hosts with 32 ranks per host
Each MPI process has 4 OpenMP threads
OMP_NUM_THREADS was set to 4
```

The output provides the following information

- The name and version of the C compiler used to build the executable
- The number of MPI ranks launched
- The number of nodes/hosts used and how many ranks are on each
- The number of threads on each rank

You may see a warning about excessive threads, this happens if `acheck` spots more threads that the number of OpenMP threads expected, these can come from the MPI library or a compiler runtime (Intel).

Run the job and verify that the output makes sense.

VERIFYING APPLICATION AFFINITY

If you add the -v command line option to acheck it will print a report on thread affinity as shown below…

```
% srun -n32 --cpus-per-task=4 --hint=multithread ./acheck-cray -v
MPI and OpenMP Affinity Checker v1.23

Built with compiler: CCE/Clang 14.0.2

There are 32 MPI Processes on 1 hosts

      Host Ranks...
 nid001078  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
           20 21 22 23 24 25 26 27 28 29 30 31
Each MPI process has 4 OpenMP threads
OMP_NUM_THREADS was set to 4
WARNING: 32 ranks had more threads per process than OpenMP threads
                     ---binding-----
      host rank thr  pinning mask
 nid001078    0   0   2 cpus 0 128
                  1   2 cpus 0 128
                  2   2 cpus 1 129
                  3   2 cpus 1 129
              1   0   2 cpus 16 144
                  1   2 cpus 16 144
                  2   2 cpus 17 145
                  3   2 cpus 17 145
                ...
```

In the output you can see which hosts each process is running on, how many OpenMP threads each process has and how many cpus the thread can run on. The final column (mask) enumerates the cpus that each thread may be scheduled to.

Using this information, you may want to attempt the following exercises by altering the job script, or reproduce example(s) from the lecture. Be aware that if the node is not fully populated it may be difficult to get the effect wanted without using cpu lists or masks.

- Check that you see the right number of ranks and threads.
    - Experiment with varying the number of processes and threads
    - Can you use all the available CPUs on the node?
    - Can you use the SMT hyperthreads or avoid using them?
    - Do the CCE and Gnu compilers behave the same? In particular if OMP_PROC_BIND is not set?
- Try running the application across more than one node.
- Try to vary how the tasks and threads are distributed.
    - Use the OpenMP binding features
    - Can you get a linear ordering through the cores?
    - What if the OMP_NUM_THREADS and the number of cpus you allow per thread don't match
- Under populating
    - Can you under populate the node keeping an even distribution to sockets?

RANK REORDERING

Note that you will not have covered this in the lectures yet, if so just try the first experiment if you have time today and come back to this later on in the week.

Using the `acheck` example provided, investigate how rank reordering affects the placement of tasks on the node.

- If you have not done so already, adjust the default job script so it runs across multiple nodes (i.e. four or more).
- Add "`export MPICH_RANK_REORDER_DISPLAY=1`" to the script.
- Run experiments changing the value of "`export MPICH_RANK_REORDER_METHOD=X`" to 0 (Round-robin), 1(default SMP) and 2 (folded). Note how this changes the hostname each rank ends up on.
- Create a custom rank-reorder file in `MPICH_RANK_ORDER` and add export `MPICH_RANK_REORDER_METHOD=3` to the run script.
    - What happens if this file specifies the wrong number of values?
    - Load the `perftools` module, see `man grid_order` for generating `MPICH_RANK_ORDER` files for Cartesian grid layouts.
- Consider how you might combine using MPMD mode and rank reordering to optimise the behaviour of applications that have individual ranks with different memory or IO requirements (E.g. IO Servers or gather/scatter models).